

The Definitive Guide to Power Query M

MODULE

6

Table Functions

Master every core Table.* function in M — column and row operations, grouping and aggregation, all join types, pivot and unpivot, FillDown, Buffer, index columns, and type transformations. Build enterprise-grade ETL pipelines that power production Power BI models.

Module 6: Table Functions

Module Contents

■ Module Overview

■ Lesson 1: Column Operations

- Table.SelectColumns and Table.RemoveColumns
- Table.RenameColumns and Table.ReorderColumns
- Table.AddColumn — The Workhorse
- Table.TransformColumns and Table.TransformColumnTypes
- Table.AddIndexColumn

■ Lesson 2: Row Operations

- Table.SelectRows — Filtering
- Table.Sort — Single and Multi-Column
- Table.Distinct — Deduplication
- Table.FirstN, Table.LastN, Table.Skip
- Table.FillDown and Table.FillUp

■ Lesson 3: Table.Group — Aggregation

- Basic Grouping
- Multiple Aggregations in One Step
- GroupKind: Global vs Local
- Custom Aggregation with each List.*

■ Lesson 4: Joining Tables

- Table.Join — All Six JoinKind Values
- Table.NestedJoin — Nested Table Pattern
- Table.Join vs Table.NestedJoin
- Self-Joins and Anti-Joins

■ Lesson 5: Pivot, Unpivot and Reshape

- Table.Pivot — Rows to Columns
- Table.Unpivot and Table.UnpivotOtherColumns
- Dynamic Pivot with List.Distinct

■ Lesson 6: Performance and Structure

- Table.Buffer — Caching for Performance
- Table.FromRecords, Table.FromColumns, Table.FromList
- Table.ExpandRecordColumn, Table.ExpandListColumn, Table.ExpandTableColumn
- Table.Combine — Append Multiple Tables

■ Lesson 7: Advanced Table Patterns for PL-300

- Pattern 1: Full ETL Pipeline in M
- Pattern 2: Self-Service Parameterised Report
- Pattern 3: Slowly Changing Dimension Detection
- Pattern 4: Dynamic Aggregation from Config

■ Demonstration 1: Multi-Table ETL Pipeline

■ Demonstration 2: Dynamic Pivot from a Fact Table

■ Lab 6: Table Functions

- Exercise 1: Column Operations

- Exercise 2: Filtering, Sorting and Deduplication
- Exercise 3: Grouping and Joining
- Exercise 4: Pivot, Unpivot and Advanced Patterns

■ Module Review and Takeaways

Module Overview

Table functions are the operational core of Power Query M. Every enterprise Power BI model — regardless of source system — is built on a chain of table transformations. This module covers the complete Table.* library in depth: the column and row manipulation functions that appear in every query, the grouping and aggregation patterns that replace database views, all six join types, pivot and unpivot for reshaping wide and tall data, and the performance functions that prevent redundant evaluation.

PL-300 exam scenarios heavily test Table.Group, Table.Join, Table.Pivot, Table.SelectRows, and Table.AddColumn. Mastery of this module — combined with the record and list knowledge from Modules 4 and 5 — covers approximately 60% of the Power Query content tested in the PL-300.

NOTE

This module assumes Modules 1–5 are complete. You should be fluent with let...in, each _, record field access, list functions, and type annotations. Module 7 (Control Flow) will deepen the conditional and error-handling patterns used here.

Before You Begin

- Complete Modules 1–5 and all lab exercises.
- Load FactSales, DimCustomer, DimProduct, DimDate, and DimRegion in Power BI Desktop.
- Download lab support files: Sales_Wide.csv, Returns.csv, and Targets.xlsx.

LESSON 1

Column Operations

After completing this lesson, you will be able to:

- Select, remove, rename, and reorder table columns.
- Add computed columns with `Table.AddColumn`.
- Transform column values and types in bulk.
- Add index columns for row-level operations.

Syntax	<code>Table.SelectColumns(table, columns, optional missingField)</code>	
Parameter	Type	Description
<code>table</code>	table	The source table.
<code>columns</code>	text or list	Column name or list of names to keep.
<code>missingField</code>	number	Optional. <code>MissingField.Error</code> (default) or <code>MissingField.Ignore</code> .
Returns	Returns a table with only the specified columns, in the specified order.	
Example	<code>Table.SelectColumns(Orders, {"OrderID", "Amount", "Region"})</code>	
Result	Table with only OrderID, Amount, Region columns	

NOTE

Column order in the output matches the order in the list — not the original table order. Use `MissingField.Ignore` to silently skip columns absent in the source.

Syntax	<code>Table.RemoveColumns(table, columns, optional missingField)</code>	
Parameter	Type	Description
<code>table</code>	table	The source table.
<code>columns</code>	text or list	Column name or list of names to drop.
Returns	Returns a table with the specified columns removed. All other columns are preserved.	
Example	<code>Table.RemoveColumns(Orders, {"TempCol", "RowNum", "Debug"})</code>	
Result	Table without TempCol, RowNum, Debug	

NOTE

Prefer `Table.RemoveColumns` over `Table.SelectColumns` when you want to keep all columns except a known set — it is resilient to new columns being added to the source.

Syntax	Table.RenameColumns(table, renames, optional missingField)	
Parameter	Type	Description
table	table	The source table.
renames	list	List of {"OldName","NewName"} pairs.
missingField	number	Optional. MissingField.Ignore skips absent columns.
Returns	Returns a table with specified columns renamed.	
Example	Table.RenameColumns(Data, {"Cust_ID", "CustomerID"}, {"rev", "Revenue"})	
Result	Columns Cust_ID and rev renamed	

NOTE

For dynamic renames from a config record, build the pairs with List.Zip: List.Zip({OldNameList, NewNameList}).

Syntax	Table.AddColumn(table, newColumnName, columnGenerator, optional columnType)	
Parameter	Type	Description
table	table	The source table.
newColumnName	text	Name for the new column.
columnGenerator	function	Row function: each _ or (row) => expression.
columnType	type	Optional. Ascribes a type to the column (e.g. type number).
Returns	Returns a table with a new column appended, whose values are computed by columnGenerator for each row. The row record is passed as the implicit _ parameter.	
Example	Table.AddColumn(Sales, "Tax", each [Amount] * 0.1, type number)	
Result	Table with new Tax column = Amount * 0.1	

NOTE

Always specify columnType when the result type is known — it prevents unnecessary type inference passes. Omitting it forces Power Query to infer the type from every value.

Common Table.AddColumn patterns:

```
// Conditional computed column
Table.AddColumn( Sales, "Category",
each if [Amount] > 10000 then "Large" else if [Amount] > 1000 then "Medium" else "Small",
type text )

// Calling a custom function per row
Table.AddColumn( Orders, "ShippingCost", each CalcShipping([Weight], [Destination]), type
number )
```

```
// Multi-field computation (see Module 5 record-expand pattern for multiple outputs)
Table.AddColumn( Inventory, "StockStatus",
each [OnHand] - [Reserved] - [BackOrder], type number )

// Referencing a lookup record (Module 5 inline lookup pattern)
Table.AddColumn( Sales, "RegionName",
each Record.FieldOrDefault(RegionRecord, [RegionCode], "Unknown"), type text )
```

Syntax	Table.TransformColumnTypes(table, typeTransformations, optional culture)	
Parameter	Type	Description
table	table	The source table.
typeTransformations	list	List of {columnName, type} pairs.
culture	text	Optional. Culture string e.g. "en-US" for date parsing.
Returns	Returns a table with specified columns cast to the given types. This is the standard final step in every ETL query.	
Example	Table.TransformColumnTypes(Data, {{"Date",type date},{"Amount",type number}})	
Result	Date and Amount columns cast to correct types	

NOTE

Apply Table.TransformColumnTypes as the last step before loading to the model. Untyped columns in Power BI default to 'any', which disables query folding and forces DAX to handle all implicit conversions — a significant performance penalty.

Syntax	Table.AddIndexColumn(table, newColumnName, optional initialValue, optional increment, optional columnType)	
Parameter	Type	Description
table	table	The source table.
newColumnName	text	Name for the index column.
initialValue	number	Optional. First index value. Default: 0.
increment	number	Optional. Step between values. Default: 1.
columnType	type	Optional. Int64.Type recommended.
Returns	Adds a sequential integer index column. Useful for row comparison, pagination, and ordering guarantees.	
Example	Table.AddIndexColumn(Orders, "RowNum", 1, 1, Int64.Type)	
Result	Table with RowNum column: 1, 2, 3, ...	

NOTE

Table.AddIndexColumn is the prerequisite for any row-comparison pattern (e.g. previous-row delta). The index gives you a stable row address: Indexed{[RowNum]-2} to access two rows back.

LESSON 2

Row Operations

After completing this lesson, you will be able to:

- Filter rows with Table.SelectRows and complex conditions.
- Sort tables by single and multiple columns.
- Remove duplicate rows with Table.Distinct.
- Propagate values down and up to fill nulls with Table.FillDown and Table.FillUp.

Syntax	Table.SelectRows(table, condition)	
Parameter	Type	Description
table	table	The source table.
condition	function	(row as record) => logical. The each _ shorthand is most common.
Returns	Returns a table containing only the rows for which condition returns true. This is the M equivalent of SQL WHERE.	
Example	Table.SelectRows(Sales, each [Amount] > 1000 and [Region] = "West")	
Result	Rows where Amount > 1000 AND Region = West	

NOTE

Table.SelectRows supports query folding when applied early in the query chain — the condition is translated to a WHERE clause by supported connectors (SQL Server, SharePoint, OData). Apply it as early as possible to maximise folding.

Common Table.SelectRows patterns:

```
// Date range filter
Table.SelectRows( Sales, each [Date] >= #date(2024,1,1) and [Date] < #date(2025,1,1) )

// Multi-value filter (list membership – does NOT fold on most connectors)
ValidRegions = {"East","West","Central","South"},
Table.SelectRows( Sales, each List.Contains(ValidRegions, [Region]) )

// Null removal – keep only rows with a non-null key
Table.SelectRows( Orders, each [CustomerID] <> null and [CustomerID] <> "" )

// Top-N per group: filter after an index (covered in Lesson 3 GroupKind pattern)
Table.SelectRows( IndexByGroup, each [GroupRank] <= 3 )
```

Syntax	Table.Sort(table, comparisonCriteria)	
Parameter	Type	Description
table	table	The source table.
comparisonCriteria	any	Column name (text) for ascending sort, or list of {columnName, Order} pairs for multi-column sort.
Returns	Returns a table sorted by the specified criteria.	
Example	Table.Sort(Sales, {{"Date", Order.Ascending}, {"Amount", Order.Descending}})	
Result	Sorted by Date ascending, then Amount descending	

NOTE

Table.Sort does NOT guarantee query folding in most connectors — the sort is performed by the M engine. For large datasets, sort in the source (SQL ORDER BY) and skip Table.Sort in M to avoid pulling all rows before sorting.

Syntax	Table.Distinct(table, optional equalityColumns)	
Parameter	Type	Description
table	table	The source table.
equalityColumns	text or list	Optional. Column(s) to determine uniqueness. If omitted, all columns are used.
Returns	Returns a table with duplicate rows removed. Keeps first occurrence.	
Example	Table.Distinct(Customers, "CustomerID")	
Result	Rows deduplicated on CustomerID	

NOTE

Passing equalityColumns is more efficient than using all columns and is required when you want uniqueness on a key column while keeping all other data from the first occurrence.

Syntax	Table.FillDown(table, columns)	
Parameter	Type	Description
table	table	The source table.
columns	list	List of column names to fill.
Returns	Replaces null values in specified columns by propagating the most recent non-null value downward. The inverse is Table.FillUp.	
Example	Table.FillDown(MergedCells, {"Category", "SubCategory"})	
Result	Nulls in Category and SubCategory replaced by last non-null value above	

NOTE

FillDown is the standard fix for Excel exports with merged cells. The cell value appears only in the top row of a merged region; all others are null. FillDown propagates that value to fill the group. Always apply FillDown before any Table.Group on the same column.

Additional row operation functions:

Function	Description	Example / Result
Table.FirstN(t,n)	Keep first n rows	Table.FirstN(Sales,100) → first 100 rows
Table.LastN(t,n)	Keep last n rows	Table.LastN(Sales,10) → last 10 rows
Table.Skip(t,n)	Skip first n rows	Table.Skip(Sales,1) → skip header row
Table.RemoveFirstN(t,n)	Remove first n rows	Same as Table.Skip
Table.RemoveLastN(t,n)	Remove last n rows	Table.RemoveLastN(t,2) → remove footers
Table.RemoveRows(t, offset, count)	Remove rows by position	Table.RemoveRows(t,5,3) → remove rows 5,6,7
Table.FillUp(t,cols)	Fill nulls upward	Propagates next non-null value upward

LESSON 3

Table.Group — Aggregation

After completing this lesson, you will be able to:

- Group a table by one or more key columns.
- Apply multiple aggregation functions in a single Table.Group call.
- Use GroupKind.Global vs GroupKind.Local.
- Build custom aggregations using each List.* inside Table.Group.

Syntax	Table.Group(table, key, aggregatedColumns, optional groupKind, optional comparer)	
Parameter	Type	Description
table	table	The source table.
key	text or list	Column name or list of column names to group by.
aggregatedColumns	list	List of {newName, aggregationFunction} pairs. Each function receives the subtable for that group.
groupKind	number	Optional. GroupKind.Global (default) or GroupKind.Local.
comparer	function	Optional. Custom equality comparer.
Returns	Returns a table with one row per unique key value. Each aggregated column is computed by applying its function to the subtable of rows sharing that key.	
Example	Table.Group(Sales, "Region", {{"TotalSales", each List.Sum([Amount])}, type number})	
Result	One row per Region with TotalSales = sum of Amount	

NOTE

GroupKind.Global (default) finds all unique key values across the entire table. GroupKind.Local groups only consecutive rows with the same key — like a run-length encoding. Use Local after sorting by the key column.

Multiple aggregations and subtable access:

```
// Multiple aggregations in one Table.Group
Table.Group(
Sales,
{"Region","ProductCategory"}, // group by two keys
{
{"TotalRevenue", each List.Sum([Revenue]), type number},
{"OrderCount", each Table.RowCount(_), Int64.Type},
{"AvgRevenue", each List.Average([Revenue]), type number},
{"MaxRevenue", each List.Max([Revenue]), type number},
{"TopProduct", each List.First(List.Sort([Product],Order.Descending)), type text},
{"SubTable", each _, type table} // keep subtable!
}
)
// The _ inside each function is the subtable for that group
```

```
// [ColumnName] inside the function extracts the column as a list from the subtable

// GroupKind.Local – use after sorting to group consecutive matching rows
Table.Group(
    Table.Sort(Sales, "Category"),
    "Category",
    {"CategoryTotal", each List.Sum([Amount]), type number}},
    GroupKind.Local
)
```

TIP

Keeping the subtable as an aggregated column (each _) creates a nested table column. You can later expand it, apply further transformations, or use it to find the top-N rows per group. This is the M equivalent of SQL window functions.

Top-N rows per group pattern using Table.Group:

```
// Get the top 3 orders by Amount for each Region
Table.Group(
    Sales,
    "Region",
    {
        {"Top3", each
            Table.FirstN(
                Table.Sort(_, {"Amount", Order.Descending})),
                3
            ),
        type table}
    }
)
// Then expand the Top3 column to get a flat table
```

LESSON 4

Joining Tables

After completing this lesson, you will be able to:

- Apply all six JoinKind values and understand their SQL equivalents.
- Use Table.Join for flat joins and Table.NestedJoin for nested table joins.
- Implement anti-joins to find non-matching rows.
- Choose between join and lookup strategies based on table size.

Syntax	Table.Join(table1, key1, table2, key2, optional joinKind, optional joinAlgorithm, optional keyEqualityComparers)	
Parameter	Type	Description
table1 / table2	table	The left and right tables to join.
key1 / key2	text or list	Join key column name(s). Lists for composite keys.
joinKind	number	JoinKind constant. See table below. Default: JoinKind.Inner.
joinAlgorithm	number	Optional. JoinAlgorithm.PairwiseHash etc.
keyEqualityComparers	list	Optional. Custom equality comparers per key column.
Returns	Returns a flat joined table. All columns from both tables are included (with suffix disambiguation if names clash).	
Example	Table.Join(Orders, "CustID", Customers, "ID", JoinKind.LeftOuter)	
Result	Left outer join: all Orders rows, matching Customer data or null	

NOTE

Table.Join produces a wide flat table immediately. Table.NestedJoin is preferred when you want to expand selectively — it avoids pulling all columns from the right table.

JoinKind	SQL Equivalent	Rows Returned
JoinKind.Inner	INNER JOIN	Only rows with matches in both tables
JoinKind.LeftOuter	LEFT OUTER JOIN	All left rows; right nulls where no match
JoinKind.RightOuter	RIGHT OUTER JOIN	All right rows; left nulls where no match
JoinKind.FullOuter	FULL OUTER JOIN	All rows from both; nulls on unmatched side
JoinKind.LeftAnti	LEFT EXCEPT / NOT IN	Left rows with NO match in right (anti-join)
JoinKind.RightAnti	RIGHT EXCEPT	Right rows with NO match in left

NOTE

JoinKind.LeftAnti is the correct way to find rows in one table that have NO corresponding row in another — the M equivalent of NOT EXISTS or NOT IN in SQL. This is the standard data quality check pattern: find orders with no matching customer.

Syntax	Table.NestedJoin(table1, key1, table2, key2, newColumnName, optional joinKind)	
Parameter	Type	Description
table1 / table2	table	The left and right tables to join.
key1 / key2	text or list	Join key column name(s).
newColumnName	text	Name for the new column that holds the nested (right) table.
joinKind	number	Optional. Default: JoinKind.LeftOuter.
Returns	Joins the tables and returns table1 with a new column containing the matching right-table rows as a nested table. You then expand only the columns you need.	
Example	Table.NestedJoin(Orders, "CustID", Customers, "ID", "CustData")	
Result	Orders with CustData column containing nested Customers rows	

NOTE

Table.NestedJoin is the ribbon UI default and is preferred for most use cases: you control exactly which right-table columns are expanded. This avoids key column collisions and pulls fewer columns, which benefits performance.

```
// Standard NestedJoin + expand pattern
let
    Joined = Table.NestedJoin( Orders, "CustID", Customers, "ID", "CustData",
        JoinKind.LeftOuter ),
    Expanded = Table.ExpandTableColumn( Joined, "CustData",
        {"CustomerName", "Email", "Segment"},
        {"CustomerName", "Email", "Segment"} ) // rename if needed
in
    Expanded

// Composite key join (match on two columns)
Table.NestedJoin(
    Sales, {"Year", "Region"},
    Targets, {"Year", "Region"},
    "TargetData", JoinKind.LeftOuter
)
```

LESSON 5

Pivot, Unpivot and Reshape

After completing this lesson, you will be able to:

- Pivot a table by turning row values into column headers with Table.Pivot.
- Unpivot wide columns into tall rows with Table.Unpivot and Table.UnpivotOtherColumns.
- Build a dynamic pivot where column values are not known at design time.

Syntax	Table.Pivot(table, pivotValues, attributeColumn, valueColumn, optional aggregationFunction)	
Parameter	Type	Description
table	table	The source table in tall (normalised) format.
pivotValues	list	The distinct values from attributeColumn that become column headers.
attributeColumn	text	The column whose values become new column headers.
valueColumn	text	The column whose values populate the new columns.
aggregationFunction	function	Optional. Default: List.Sum. Use when multiple rows share the same key + attribute combination.
Returns	Returns a wide table with one new column per unique value in attributeColumn.	
Example	Table.Pivot(Data, {"Q1","Q2","Q3","Q4"}, "Quarter", "Sales", List.Sum)	
Result	Wide table: one column per quarter with summed sales	

NOTE

pivotValues must be a literal list. For a dynamic pivot where values are not known in advance, use List.Distinct(table[attributeColumn]) to build the list at runtime (see the dynamic pivot pattern in Lesson 7).

Syntax	Table.Unpivot(table, pivotColumns, attributeColumn, valueColumn)	
Parameter	Type	Description
table	table	The source table in wide format.
pivotColumns	list	The column names to unpivot (turn into rows).
attributeColumn	text	Name for the new column that will hold original column names.
valueColumn	text	Name for the new column that will hold the values.
Returns	Converts specified wide columns into a tall attribute-value pair format. The inverse of Table.Pivot.	
Example	Table.Unpivot(Sales, {"Q1","Q2","Q3","Q4"}, "Quarter", "Revenue")	
Result	Tall table: rows for each Quarter/Revenue combination	

NOTE

Use Table.Unpivot when you know exactly which columns to unpivot. Use Table.UnpivotOtherColumns when you know which columns to KEEP and want to unpivot everything else.

Syntax	Table.UnpivotOtherColumns(table, pivotColumns, attributeColumn, valueColumn)	
Parameter	Type	Description
table	table	The source table.
pivotColumns	list	Columns to KEEP fixed (not unpivoted).
attributeColumn	text	Name for the attribute column.
valueColumn	text	Name for the value column.
Returns	Unpivots all columns EXCEPT those specified in pivotColumns. Robust to new columns being added to the source.	
Example	Table.UnpivotOtherColumns(Data, {"Region","ProductID"}, "Month", "Sales")	
Result	All columns except Region and ProductID unpivoted into Month/Sales rows	

NOTE

Table.UnpivotOtherColumns is the safer choice for Excel or CSV sources that may gain new columns (e.g. new months) over time. It automatically unpivots any new column without requiring code changes.

Dynamic pivot — pivot values not known at design time:

```
// Source: tall table with Region, Month, Sales columns
// We do not know which months exist – build the pivot list dynamically

let
    Source = SalesNormalised,
    PivotValues = List.Sort( List.Distinct( Source[Month] ) ), // dynamic list
```

```
Pivoted = Table.Pivot( Source, PivotValues, "Month", "Sales", List.Sum )  
in  
Pivoted  
// New months in the data automatically create new columns – no code change needed
```

LESSON 6

Performance and Structure

After completing this lesson, you will be able to:

- Use Table.Buffer to prevent repeated table evaluation.
- Construct tables from records, columns, and lists.
- Expand nested record, list, and table columns.
- Append multiple tables with Table.Combine.

Syntax	Table.Buffer(table, optional options)	
Parameter	Type	Description
table	table	The source table to cache in memory.
options	record	Optional. Currently unused but reserved for future options.
Returns	Forces the table to be fully evaluated and held in memory. Subsequent references to the buffered table do not re-evaluate from the source.	
Example	let Buffered = Table.Buffer(ExpensiveQuery) in Buffered	
Result	ExpensiveQuery evaluated once, held in RAM	

NOTE

Use Table.Buffer when a query result is referenced multiple times in the same let...in expression (e.g. in both a join and a filter). Without buffering, M may re-evaluate the full source query for each reference — doubling or tripling refresh time. Do NOT buffer tables that are only referenced once.

Table construction functions:

Function	Description	Result / Note
Table.FromRecords(records)	List of records → table	Each record becomes a row
Table.FromColumns(lists, cols)	Parallel lists → table	Each list becomes a column
Table.FromList(list, splitter, cols)	List → table	Single list split into columns
Table.FromRows(rows, cols)	List of value lists → table	Each inner list is a row
Table.Combine(tables)	List of tables → one table	Appends tables vertically (UNION ALL)
Table.ColumnNames(t)	Table → list of names	Returns list of column name strings
Table.RowCount(t)	Table → integer	Number of rows; also List.Count(t[Col])

Expand functions for nested columns:

Function	Description	Typical Use
<code>Table.ExpandRecordColumn(t, col, fields)</code>	[Record] column → multiple columns	Flatten nested records (JSON APIs)
<code>Table.ExpandListColumn(t, col)</code>	[List] column → one row per item	Explode list cells into rows
<code>Table.ExpandTableColumn(t, col, fields)</code>	[Table] column → flat table	After <code>Table.NestedJoin</code>

TIP

`Table.Combine` requires all tables to have compatible schemas. Use `Table.TransformColumnTypes` on each table before combining, and consider `Table.SelectColumns` to enforce the same column set. A mismatch causes a 'Column not found' error at refresh.

LESSON 7

Advanced Table Patterns for PL-300

After completing this lesson, you will be able to:

- Build a complete M ETL pipeline from raw source to typed model table.
- Implement dynamic grouping driven by a config record.
- Detect slowly changing dimension changes between two snapshots.
- Apply an anti-join for data quality validation.

Pattern 1: Full ETL Pipeline in M

```
// Complete pipeline: raw CSV → typed, cleaned, enriched model table
let
// 1. Ingest
Source = Csv.Document(File.Contents("Sales.csv"), [Delimiter=","]),
Promoted = Table.PromoteHeaders(Source),

// 2. Rename to canonical names
Renamed = Table.RenameColumns(Promoted,
{{"sale_dt", "Date"}, {"cust_id", "CustomerID"},
{"rev_usd", "Revenue"}, {"rgn_cd", "Region"}}),

// 3. Type
Typed = Table.TransformColumnTypes(Renamed,
{{"Date", type date}, {"CustomerID", type text},
{"Revenue", type number}, {"Region", type text}}),

// 4. Clean
NoNulls = Table.SelectRows(Typed, each [CustomerID] <> null),
Filled = Table.FillDown(NoNulls, {"Region"}),

// 5. Enrich
Indexed = Table.AddIndexColumn(Filled, "RowID", 1, 1, Int64.Type),
WithTax = Table.AddColumn(Indexed, "Tax", each [Revenue]*0.1, type number),

// 6. Lookup join
Joined = Table.NestedJoin(WithTax, "Region", DimRegion, "Code",
"RegData", JoinKind.LeftOuter),
Expanded = Table.ExpandTableColumn(Joined, "RegData",
{"FullName"}, {"RegionName"}),

// 7. Final column selection
Result = Table.SelectColumns(Expanded,
{"RowID", "Date", "CustomerID", "Revenue", "Tax", "Region", "RegionName"})
in
Result
```

Pattern 2: Slowly Changing Dimension Detection

```
// Detect which customer records changed between two snapshots
let
// Buffer both tables – they are referenced twice each
Snapshot1 = Table.Buffer(Yesterday),
Snapshot2 = Table.Buffer(Today),
```

```
// Step 1: Find new keys (in Today but not Yesterday)
NewKeys = Table.Join(Snapshot2, "CustomerID", Snapshot1, "CustomerID",
JoinKind.LeftAnti),

// Step 2: Find deleted keys (in Yesterday but not Today)
DeletedKeys= Table.Join(Snapshot1, "CustomerID", Snapshot2, "CustomerID",
JoinKind.LeftAnti),

// Step 3: Find changed rows (in both but with different values)
InBoth = Table.Join(Snapshot2, "CustomerID", Snapshot1, "CustomerID",
JoinKind.Inner),
Changed = Table.SelectRows(InBoth, each [Revenue] <> [Revenue.1]
or [Segment] <> [Segment.1])
in
{ Table.RowCount(NewKeys), Table.RowCount(DeletedKeys), Table.RowCount(Changed) }
// Result: {5, 2, 14} – 5 new, 2 deleted, 14 changed
```

Pattern 3: Dynamic Aggregation from Config

```
// A config record drives which column to aggregate and by what method
AggConfig = [ GroupBy="Region", AggCol="Revenue", Method="Sum" ],

// Build the aggregation function dynamically
AggFn = if AggConfig[Method] = "Sum" then (t) => List.Sum(t[AggConfig[AggCol]])
else if AggConfig[Method] = "Avg" then (t) => List.Average(t[AggConfig[AggCol]])
else if AggConfig[Method] = "Max" then (t) => List.Max(t[AggConfig[AggCol]])
else error "Unknown method: " & AggConfig[Method],

Result = Table.Group(
Sales,
AggConfig[GroupBy],
{{ AggConfig[Method] & "_" & AggConfig[AggCol], AggFn, type number }}
)
```

DEMONSTRATION 1

Multi-Table ETL Pipeline

Scenario: Build a complete sales model from five raw CSV files: FactSales, DimCustomer, DimProduct, DimDate, and DimRegion. Apply the full pipeline: ingest, rename, type, clean, join, enrich, group, and validate.

Demonstration Steps

1	Load all five CSV files. Show the raw data — untyped columns, inconsistent names, nulls in key columns, merged-cell nulls in Region.
2	Apply Table.RenameColumns and Table.TransformColumnTypes to each table. Show how TypeMap records from Module 5 simplify this step.
3	Apply Table.FillDown to the Region column in FactSales. Show before and after.
4	Join FactSales to DimCustomer using Table.NestedJoin + Table.ExpandTableColumn. Use JoinKind.LeftAnti first to check for orphaned CustomerIDs.
5	Apply Table.Group to produce a Region × ProductCategory summary table with TotalRevenue, OrderCount, and AvgRevenue in one step.
6	Apply dynamic Table.Pivot to turn the Month column into wide format. Build the pivot list dynamically with List.Distinct.
7	Add Table.Buffer to the DimCustomer query and show the refresh time difference when the same table is referenced in two join steps.

RESULT

A fully typed, cleaned, joined, and summarised model with five related tables loaded into the Power BI model. Query folding is active on the FactSales source for all row filters and column selections.

DEMONSTRATION 2

Dynamic Pivot from a Fact Table

Scenario: Build a monthly revenue matrix (Region × Month) where the month columns are fully dynamic — new months appear automatically without code changes.

Demonstration Steps

1	Start from the FactSales table. Show that it is in tall (normalised) format: one row per transaction with Date, Region, and Revenue columns.
2	Add a MonthLabel column: <code>Table.AddColumn(Source, "Month", each Date.ToText([Date], "MMM-yyyy"), type text)</code> .
3	Build the pivot list dynamically: <code>PivotCols = List.Sort(List.Distinct(Source[Month]))</code> .
4	Apply <code>Table.Pivot</code> using <code>PivotCols</code> as <code>pivotValues</code> , "Month" as <code>attributeColumn</code> , "Revenue" as <code>valueColumn</code> , and <code>List.Sum</code> as the aggregation function.
5	Sort the resulting columns using <code>Table.ReorderColumns</code> with the dynamic list.
6	Add new test data to the source with a future month. Refresh. Show the new month column appearing automatically without any code change.
7	Discuss: why static pivot lists are an anti-pattern for calendar-based data.

RESULT

A Region × Month revenue matrix that automatically adds new month columns as data arrives. No manual updates to the query are required when a new reporting period begins.

LAB 6

Table Functions

In this lab you apply the complete Table.* library to build a production-grade sales model. Use FactSales, DimCustomer, DimProduct, DimRegion, and the provided Sales_Wide.csv and Returns.csv files.

Lab Scenario

Contoso Analytics requires a fully transformed sales model. You must: clean and type all source tables, build a dimension join, produce a regional summary with Table.Group, unpivot the wide Sales file, and validate data quality with anti-joins.

NOTE

All exercises use M only — no ribbon clicks. Write all code in the Advanced Editor. Label every step. Complete exercises in order.

Exercise 1: Column Operations (Estimated time: 20 minutes)

Apply column operations to clean and standardise the raw FactSales source.

- Load FactSales. Apply Table.PromoteHeaders if the first row contains headers.
- Rename all columns to canonical names using a ColumnMap record and List.Zip (Module 5 pattern).
- Apply Table.TransformColumnTypes using a TypeMap record.
- Add a Revenue column: Table.AddColumn(..., "Revenue", each [Quantity]*[UnitPrice], type number).
- Add a RowID index: Table.AddIndexColumn(..., "RowID", 1, 1, Int64.Type).
- Use Table.SelectColumns to keep only the columns needed for the model.
- STRETCH: Add a YearMonth column using Date.Year and Date.Month combined into a text key.

RESULT

FactSales has canonical column names, correct types, Revenue, RowID, and only model-relevant columns. All steps are named descriptively.

Exercise 2: Filtering, Sorting and FillDown (Estimated time: 20 minutes)

- Apply Table.FillDown to the Region and SubRegion columns (simulate merged-cell export).
- Filter to a date range: keep only 2023 and 2024 data using Table.SelectRows.
- Remove rows where CustomerID is null using Table.SelectRows.
- Sort by Date ascending then Revenue descending using Table.Sort with a list of criteria.
- Apply Table.Distinct on CustomerID to build a unique customer list for DimCustomer.
- STRETCH: Find the top 5 Revenue rows per Region using Table.Group with a sorted subtable.

RESULT

FactSales is filtered to 2023-2024, null-free, and sorted. A distinct customer list is available as a staging query.

Exercise 3: Grouping and Joining (Estimated time: 25 minutes)

- Apply Table.Group to FactSales grouped by Region and ProductCategory, computing TotalRevenue, OrderCount, and AvgRevenue.

- Join FactSales to DimCustomer using Table.NestedJoin (LeftOuter on CustomerID). Expand CustomerName and Segment only.
- Join FactSales to Returns using JoinKind.LeftAnti to find orders with no return. Count the result with Table.RowCount.
- Join Returns to FactSales using JoinKind.Inner to find returned orders. Add a ReturnRate column: returns / total orders per region.
- Buffer DimCustomer before the join. Compare refresh time with and without Table.Buffer.
- STRETCH: Implement a composite key join on Year + Region between FactSales and Targets.

RESULT

A grouped regional summary table and a joined FactSales with customer segments. Anti-join identifies orders with no returns. Buffered DimCustomer reduces refresh time when referenced in multiple join steps.

Exercise 4: Pivot, Unpivot and Advanced Patterns (Estimated time: 25 minutes)

- Load Sales_Wide.csv. It has columns: Region, Jan, Feb, Mar, ..., Dec (monthly revenue).
- Unpivot all month columns using Table.UnpivotOtherColumns (keep Region fixed). Name the new columns Month and Revenue.
- From the unpivoted table, apply a dynamic Table.Pivot: build PivotValues with List.Distinct, pivot Month back to wide format by Region.
- Apply the full ETL pipeline from Pattern 1 to FactSales in a single let...in expression.
- Implement the SCD detection pattern from Pattern 2 using DimCustomer vs a provided snapshot table. Report counts of new, deleted, and changed rows.
- STRETCH: Build the dynamic aggregation config pattern (Pattern 3). Test with Method='Sum', 'Avg', and 'Max'.

RESULT

Sales_Wide is unpivoted and re-pivoted dynamically. The full ETL pipeline runs as a single clean query. SCD detection correctly identifies changed customer records. Dynamic aggregation config produces correct results for all three methods.

Module Review and Takeaways

Review Questions

What is the difference between `Table.SelectColumns` and `Table.RemoveColumns`, and when should you prefer each?

Answer: `Table.SelectColumns` keeps only named columns in the specified order — useful when you want a fixed output schema. `Table.RemoveColumns` drops named columns and keeps everything else — safer when the source may gain new columns you want to retain automatically. Prefer `RemoveColumns` when new source columns are expected.

What does `GroupKind.Local` do differently from `GroupKind.Global`?

Answer: `GroupKind.Global` (default) scans the entire table and groups all rows with the same key, regardless of position. `GroupKind.Local` groups only consecutive rows sharing the same key — it stops a group when the key changes. Local requires the table to be pre-sorted by the key column and is faster but only correct after sorting.

What is the difference between `Table.Join` and `Table.NestedJoin`?

Answer: `Table.Join` produces a flat wide table immediately — all columns from both tables merged. `Table.NestedJoin` adds a column containing the matching right-table rows as a nested table, which you then expand selectively. `NestedJoin` gives you control over which right-table columns to pull and avoids key column name collisions.

How do you perform an anti-join in M?

Answer: Use `JoinKind.LeftAnti` in `Table.Join` or `Table.NestedJoin`. Left anti-join returns all rows from the left table that have NO matching row in the right table. This is the M equivalent of SQL NOT EXISTS or NOT IN — essential for data quality checks.

What is `Table.Buffer` and when should you use it?

Answer: `Table.Buffer` forces a table to be fully evaluated once and held in memory. Use it when a query result is referenced multiple times in the same let...in expression — for example, when the same staging table is used in both a join and a count. Without buffering, M may re-evaluate from the source for each reference.

What is the difference between `Table.Unpivot` and `Table.UnpivotOtherColumns`?

Answer: `Table.Unpivot` requires you to specify which columns to unpivot. `Table.UnpivotOtherColumns` requires you to specify which columns to KEEP fixed — everything else is unpivoted. Use `UnpivotOtherColumns` when the number of columns to unpivot may change over time (e.g. new monthly columns in an Excel export).

Why should `Table.TransformColumnTypes` be applied as the last step in an ETL pipeline?

Answer: Applying types early causes Power Query to convert values during each intermediate step, which can cause errors or unexpected coercions. Applying it last ensures all text manipulation and cleaning is done first, then types are assigned once. In the Power BI model, untyped columns prevent query folding and force DAX to handle conversions.

How does dynamic pivoting work and why is a static pivot list an anti-pattern?

Answer: Dynamic pivoting builds the `pivotValues` list at runtime using `List.Distinct(table[attributeColumn])`. A static list hard-codes values like {"Jan","Feb",...} — the query breaks when a new value appears. Dynamic

pivoting automatically creates new columns for new values with no code change.

Best Practices

- Apply `Table.SelectRows` as early as possible in the query chain to maximise query folding. Filtering at the source reduces the rows the M engine must process.
- Always specify the `columnType` parameter in `Table.AddColumn` when the type is known. Omitting it forces a type inference pass over every value.
- Use `Table.NestedJoin` over `Table.Join` in most cases — it gives you explicit control over which right-table columns to expand and avoids column name collisions.
- Apply `Table.FillDown` before any `Table.Group` on the same column. Null key values cause a separate null group that is almost always unintended.
- Prefer `Table.RemoveColumns` over `Table.SelectColumns` when new source columns should be carried through automatically. `Table.SelectColumns` silently drops any column not listed.
- Use `Table.Buffer` on staging tables that are referenced more than once in a query. Measure the impact — buffering large tables consumes RAM and may not always improve speed.
- Use `Table.UnpivotOtherColumns` instead of `Table.Unpivot` for calendar-based wide tables so new month columns are handled automatically.
- Always apply `Table.TransformColumnTypes` as the final step, after all cleaning and enrichment. Type columns in the model, not in intermediate steps.
- For dynamic pivots, always sort the pivot value list with `List.Sort` before passing it to `Table.Pivot`. This ensures consistent column ordering across refreshes.
- Use `JoinKind.LeftAnti` as your first data quality check when joining to a dimension — identify orphaned fact rows before they silently produce nulls in the model.

Common Issues and Solutions

Column not found after `Table.Combine`

Cause: The tables being combined have different column sets — possibly different names or cases.

Solution: Apply `Table.SelectColumns` with the same explicit list to each table before `Table.Combine`. Or use `Table.TransformColumnTypes` on each to ensure identical schemas.

`Table.Group` produces a null group unexpectedly

Cause: The key column contains null values. Each null is treated as a distinct group key.

Solution: Apply `Table.SelectRows` to remove null keys before grouping, or apply `Table.FillDown` if the nulls result from merged cells.

`Table.Join` creates duplicate column names

Cause: Both tables have columns with the same name (e.g. both have a 'Date' column). `Table.Join` appends a suffix (.1, .2) but this is hard to read.

Solution: Use `Table.NestedJoin` and expand only the columns you need with explicit rename. Alternatively, remove or rename conflicting columns before joining.

Dynamic pivot fails with 'Column names must be distinct'

Cause: The attribute column contains duplicate values for the same key, so the pivot produces multiple values for one cell without an aggregation function.

Solution: Always specify an aggregation function (e.g. List.Sum) as the fifth argument to Table.Pivot. Without it, the function defaults to List.Last and fails on duplicates.

Table.Buffer does not improve performance

Cause: The table is only referenced once, so buffering adds overhead without benefit. Or the bottleneck is query folding, not re-evaluation.

Solution: Only buffer tables referenced multiple times. Check query folding first with the Performance Analyzer — if folding is active, the source handles the query and buffering may not help.

Key Functions — Module 6 Quick Reference

All table functions introduced in this module, organised by category.

Function	Category	Key Behaviour
<code>Table.SelectColumns(t, cols)</code>	Column	Keep only specified columns in listed order
<code>Table.RemoveColumns(t, cols)</code>	Column	Drop specified columns; keep all others
<code>Table.RenameColumns(t, pairs)</code>	Column	Rename columns by {old,new} pairs
<code>Table.ReorderColumns(t, cols)</code>	Column	Reorder columns to specified order
<code>Table.AddColumn(t, name, fn, type)</code>	Column	Add computed column via row function
<code>Table.TransformColumns(t, ops)</code>	Column	Apply functions to specified column values
<code>Table.TransformColumnTypes(t, pairs)</code>	Column	Cast columns to specified M types
<code>Table.AddIndexColumn(t, name, start, step)</code>	Column	Add sequential integer index
<code>Table.SelectRows(t, condition)</code>	Row	Keep rows where condition is true (SQL WHERE)
<code>Table.Sort(t, criteria)</code>	Row	Sort by column(s) ascending or descending
<code>Table.Distinct(t, cols)</code>	Row	Remove duplicate rows (optionally by key cols)
<code>Table.FirstN(t, n)</code>	Row	Keep first n rows
<code>Table.LastN(t, n)</code>	Row	Keep last n rows
<code>Table.Skip(t, n)</code>	Row	Skip first n rows
<code>Table.FillDown(t, cols)</code>	Row	Propagate non-null values downward
<code>Table.FillUp(t, cols)</code>	Row	Propagate non-null values upward
<code>Table.Group(t, key, aggs)</code>	Aggregate	Group by key; apply aggregation functions
<code>Table.Join(t1, k1, t2, k2, kind)</code>	Join	Flat join with JoinKind constant
<code>Table.NestedJoin(t1, k1, t2, k2, col)</code>	Join	Join with right rows as nested table column
<code>Table.Pivot(t, vals, attrCol, valCol)</code>	Reshape	Tall → wide; row values become columns
<code>Table.Unpivot(t, cols, attr, val)</code>	Reshape	Wide → tall; columns become rows
<code>Table.UnpivotOtherColumns(t, keep, attr, val)</code>	Reshape	Unpivot all except specified columns
<code>Table.Buffer(t)</code>	Performance	Cache table in memory; prevent re-evaluation
<code>Table.Combine(tables)</code>	Structure	Append list of tables (UNION ALL)
<code>Table.FromRecords(records)</code>	Structure	List of records → table
<code>Table.FromColumns(lists, cols)</code>	Structure	Parallel lists → table
<code>Table.ExpandRecordColumn(t, col, fields)</code>	Expand	Flatten nested [Record] column
<code>Table.ExpandTableColumn(t, col, fields)</code>	Expand	Flatten nested [Table] column
<code>Table.ExpandListColumn(t, col)</code>	Expand	Explode nested [List] column into rows
<code>Table.RowCount(t)</code>	Inspect	Number of rows in table
<code>Table.ColumnNames(t)</code>	Inspect	List of column name strings

Module Summary

- Column operations (`SelectColumns`, `RemoveColumns`, `RenameColumns`, `AddColumn`, `TransformColumnTypes`, `AddIndexColumn`) are the building blocks of every M query. Always specify types and apply `TransformColumnTypes` as the last step.
- Row operations (`SelectRows`, `Sort`, `Distinct`, `FillDown`) clean and shape data before aggregation. Apply `SelectRows` early for query folding; apply `FillDown` before `Group`.
- `Table.Group` aggregates by any number of keys in a single step. The aggregation function receives the subtable for each group — enabling complex subtable-level logic. `GroupKind.Local` groups consecutive rows only.
- `Table.Join` and `Table.NestedJoin` implement all six SQL join types. Prefer `NestedJoin` for selective column expansion. Use `JoinKind.LeftAnti` for anti-joins and data quality checks.
- `Table.Pivot` turns row values into columns; `Table.UnpivotOtherColumns` turns columns into rows. Build the pivot list dynamically to handle new values automatically.
- `Table.Buffer` prevents re-evaluation of tables referenced multiple times. `Table.Combine` appends tables. Expand functions flatten nested record, list, and table columns.
- Four advanced patterns: full ETL pipeline, SCD change detection, dynamic aggregation, and parameterised self-service reporting — all driven by `Table.*` functions.

NOTE

Next Module — Module 7: Control Flow and Advanced M. Covers `if/then/else`, `let...in` deep dive, `each _` mechanics, `try...otherwise`, error handling, function definitions, and recursive functions with `@`. This module ties every previous concept together into reusable, defensive, production-grade M code.